

Addressable Memory: A Lossless Primitive for Sequential Knowledge Editing in Frozen Language Models

Mohamed Mabrok
Minds-R-Lab

May 24, 2026

Abstract

Adding new factual knowledge to a deployed language model is the most basic form of learning a database can do — and yet every weight-modification method we know (sequential fine-tuning, LoRA, and rank-one knowledge editors like ROME / MEMIT) destroys prior knowledge as the number of edits grows. We argue this is not a tuning problem but a structural one: facts written into shared parameters necessarily compete with the parameters’ existing function. We propose an alternative primitive — an *addressable memory layer* — that wraps a single mid-stack MLP and stores each new fact as a $(k, \delta v)$ pair in an external bank, leaving every base weight untouched. The key k is the MLP intermediate at the last prompt token captured during a teacher-forced insertion pass; the value δv is a gradient-optimized perturbation that drives the language modeling head to the target. At inference, cosine similarity over keys gates which slot fires per position. We introduce SFIB, a pre-registered benchmark for sequential one-shot fact insertion over a synthetic knowledge base of 2,000 pretrain triples and 500 insertion triples with fictional syllable-pool entities. On GPT-2 small (124M), with 3-seed validation, our primitive achieves $\text{Insertion@500} = 0.468 \pm 0.030$ and $\text{Retention@500} = 0.922 \pm 0.033$ — $3.2\times$ MEMIT on insertion and $5.9\times$ on retention, simultaneously, over 500 sequential edits. We further extend the primitive with multi-position v^* -optimization to handle multi-token target answers under modern BPE tokenizers, and demonstrate on Qwen2.5-0.5B that the architecture transfers to a different family (SwiGLU MLP, RoPE, GQA) achieving $\text{Insertion@500} = 0.995$ at $\text{Retention@500} = 0.811$ — the only weight-modification method we tested that preserves more than 5% of prior knowledge under this stream.

1 Introduction

A language model that cannot learn a new fact after deployment is, in a narrow but important sense, less capable than a database. Databases insert records in $O(1)$ time without disturbing other records. Language models require gradient descent through tens to hundreds of billions of parameters to absorb a single new fact, and the gradient invariably disturbs neighboring memories. This trade-off has been the central technical obstacle to continual learning for as long as neural networks have existed, and it has shaped a generation of workarounds — replay buffers, parameter freezing, low-rank adapters, retrieval-augmented generation, and explicit knowledge editors that try to make minimal weight changes per fact.

The state-of-the-art editor at the time of writing is the MEMIT family of methods meng2022rome, meng2022m which uses causal tracing to localize a single fact to a particular mid-stack MLP and then applies a rank-one update to that MLP’s down-projection. When a single fact is inserted in isolation, MEMIT can achieve 99% efficacy with negligible collateral. The original paper reports successful editing of thousands of facts in one batched update. Yet *sequential* insertion — the realistic deployed setting where facts arrive one at a time over the life of a model — shows a different

picture. Each rank-one update overlaps in support with all previous updates, the perturbations accumulate in the same low-dimensional subspaces of MLP weight space, and accuracy on both the new facts and on the old pretrained knowledge degrades together.

We argue that this degradation is fundamental, not technical. Any method that writes new information into a frozen-but-overwritable parameter set must spend that parameter set’s degrees of freedom; eventually the parameters cannot simultaneously represent everything they used to plus everything they are now asked to know. The only way around this is to *not write into the existing parameters at all* — to instead maintain new information in a separately-addressable structure that the model can consult without disturbing its prior weights.

This idea is not new in form. Memory-augmented neural networks date back at least to the Neural Turing Machine graves2014ntm and the Differentiable Neural Computer graves2016dnc; Memory Networks weston2015memnets and Modern Hopfield Networks ramsauer2020modern explore variants of the same theme. Most recently, berges2024memory scaled product-key memory layers to trillion-token training budgets and showed they outperform dense FFN layers on knowledge-intensive evaluation. What has been less explored is the specific case of *post-training, one-shot, sequential insertion* on a frozen pretrained backbone, where the question is not whether addressable memory can help during pretraining but whether it can serve as the alternative to gradient-based editors like MEMIT after a model is already in deployment. This is the regime we study.

Our contributions are four:

1. We introduce the **Sequential Fact Insertion Benchmark (SFIB)**: a pre-registered synthetic knowledge base with 2,000 pretrain triples, 500 insertion triples (disjoint entities), and 200 multi-hop composition queries. SFIB measures three quantities at every checkpoint N along the insertion stream — Ins@ N (accuracy on the inserted facts), Ret@ N (accuracy on the pretrained facts), and Com@ N (accuracy on two-hop questions combining one old fact and one new fact). Entities are constructed from fictional syllable pools to guarantee no overlap with the model’s pretraining distribution.
2. We propose the **addressable memory primitive**: a drop-in wrapper around a chosen mid-stack MLP that stores each new fact as a $(k, \delta v)$ slot in an external key-value bank. At forward pass time, cosine similarity between the layer’s intermediate state and stored keys selects which slot’s δv to add to the MLP output. No base weight is ever modified. The primitive is family-agnostic: we test it on GPT-2’s vanilla MLP and on Qwen2.5’s SwiGLU MLP using the same code path.
3. We provide **architectural ablations** showing the contribution of each design choice. The most important is the firing rule: when slots fire at *every* sequence position rather than only at the next-token prediction position, accuracy is capped because spurious slot firings at upstream tokens pollute the prediction. Restricting firing to the last position of each forward pass — which by construction is where the language-modeling head reads from — recovers an order-of-magnitude improvement in Insertion@ N at small N .
4. We show the architecture **transfers across model families and target tokenization regimes**. With a single-position v^* optimization, the primitive works perfectly when target answers tokenize to a single subword (as is common under GPT-2’s BPE on our synthetic nouns). For target answers that BPE-tokenize into multiple subwords (common under Qwen2’s tokenizer), single-position perturbation controls only the first generated token. We extend the primitive with *multi-position v^* -optimization*: one $(k, \delta v)$ slot is captured and optimized per predicting position in the prompt-plus-target sequence. At eval, the wrapped MLP cosine-matches each generated token’s intermediate state against all stored keys, and the slot for that position fires deterministically. With this extension Qwen2.5-0.5B achieves Insertion@500 = 0.995.

The headline number on GPT-2 small with three-seed validation is $\text{Ins@500} = 0.468 \pm 0.030$, $\text{Ret@500} = 0.922 \pm 0.033$. The strongest baseline (MEMIT, with the same v^* optimization infrastructure but the rank-one weight update kept in place) reaches $\text{Ins@500} = 0.145 \pm 0.031$, $\text{Ret@500} = 0.186 \pm 0.073$. The next-strongest (sequential fine-tuning at the canonical learning rate) reaches $\text{Ins@500} = 0.118$, $\text{Ret@500} = 0.079$. Across both axes simultaneously, no weight-modification baseline tested comes within a factor of three of the addressable memory primitive.

The remainder of the paper is organized as follows. Section 2 reviews related work in knowledge editing and memory-augmented architectures. Section 3 reviews the relevant background on how transformer MLPs store and retrieve factual associations. Section 4 introduces the addressable memory primitive formally and gives the forward and insertion algorithms. Section 5 describes the SFIB benchmark. Section 6 lays out the experimental protocol. Section 7 reports the main results, ablations, and the multi-architecture transfer. Section 8 discusses limitations and connections to related approaches. Section 9 concludes.

2 Related Work

Locate-and-edit knowledge editors. The line of work directly comparable to ours is the localize-and-edit family: **ROME** meng2022rome performs a single rank-one update to a chosen MLP’s down-projection to inject one fact at a time, guided by causal tracing of where the model stores subject information. **MEMIT** meng2022memit extends ROME to batched updates through a least-squares optimum across many edits at once, scaling to thousands of facts in a single optimization step. Both rely on a v^* optimization that finds a target MLP-output vector reproducing the desired prediction, then expresses the required parameter change in closed form. We use the same v^* -optimization machinery (Adam steps on a perturbation vector with L_2 regularization and norm constraint) but *store* the result in an external bank instead of dissolving it into the weights. Subsequent comparisons hartvigsen2023grace, hase2023doesedits have noted that locate-and-edit methods compound destructively under sequential application, motivating the present work.

Memory-augmented architectures. The Neural Turing Machine graves2014ntm, the Differentiable Neural Computer graves2016dnc, and the broader Memory Network weston2015memnets family established that recurrent computation with explicit addressable storage can solve tasks — algorithmic copy, sort, associative recall — that are difficult for vanilla recurrence. Modern Hopfield Networks ramsauer2020modern showed that exponentially-large autoassociative storage can be implemented with a small differentiable energy function. Most recent and most directly relevant is **Memory Layers at Scale** berges2024memory, which replaces a transformer FFN with a learned product-key memory bank; at trillion-token training scales, knowledge-intensive evaluation accuracy improves substantially over the dense baseline. Our work differs in that we are not training a new model: we wrap a single pre-existing MLP layer of a frozen pretrained model and use the wrapper only for sequential post-training fact insertion. The memory bank is populated by a deterministic insertion procedure rather than by joint training with the base model.

Knowledge in transformer MLPs. geva2021keyvalue interprets transformer FFN layers as implicit key-value memories where the first projection’s rows act as keys and the second projection’s columns as values. This framing motivated ROME’s choice of editing the MLP down-projection, and it motivates our choice of MLP intermediate (the activated input to the down-projection) as our key representation. The crucial difference from geva2021keyvalue is that we make the key-value memory *external and explicit* rather than implicit in the weight matrix.

Parameter-efficient fine-tuning. LoRA hu2022lora and adapter-style methods constrain updates to low-rank or low-parameter subspaces. They reduce the volume of weight modifi-

cation per fact but do not change the fundamental property that new information is written to shared parameters. We confirm this on SFIB: LoRA-sequential at tuned hyperparameters reaches $\text{Ins@500} = 0.008$ and $\text{Ret@500} = 0.004$ — worse than full sequential fine-tuning under the same conditions. The cause is that forcing each fact’s gradient into a few hundred thousand shared adapter parameters concentrates the perturbation rather than distributing it; the catastrophe arrives sooner, not later.

Activation steering and controller families. A complementary line of recent work intervenes on the residual stream rather than the weights. **NTK-Mirror** chlon2026ntkmirror learns sparse signed log-gates on residual-stream channels and composes them across "controller memories" for different documents or task styles. The mechanism is multiplicative rather than additive and operates per-channel rather than per-position. The author explicitly scopes the method out of factual insertion — "for facts, use prompt/RAG provenance" — so the present work occupies a complementary slot in design space: per-fact, additive at one MLP layer, with deterministic insertion via v^* -optimization, targeting the factual case the controller family declines.

3 Background: Knowledge in Transformer MLPs

A standard pre-norm decoder block of a transformer comprises an attention sublayer and an MLP sublayer, each wrapped in a residual connection:

$$x_\ell^{(\text{mid})} = x_\ell + \text{Attn}(\text{LN}(x_\ell)), \quad (1)$$

$$x_{\ell+1} = x_\ell^{(\text{mid})} + \text{MLP}(\text{LN}(x_\ell^{(\text{mid})})). \quad (2)$$

The MLP can take one of two forms depending on the architecture family. For GPT-2 radford2019gpt2 and other classical decoders:

$$\text{MLP}_{\text{vanilla}}(x) = c_{\text{proj}}(\sigma(c_{\text{fc}}(x))), \quad (3)$$

where c_{fc} and c_{proj} are 1-D convolutions (equivalent to linear layers up to a transpose), and σ is a nonlinearity (GeLU for GPT-2). For Llama touvron2023llama, Qwen2 yang2024qwen2, and other modern SwiGLU-family decoders:

$$\text{MLP}_{\text{SwiGLU}}(x) = W_{\text{down}}[\sigma(W_{\text{gate}}x) \odot W_{\text{up}}x], \quad (4)$$

where $\odot$ is elementwise product and σ is typically SiLU.

In both forms, the MLP’s output at a given position t depends only on the hidden state at position t , computed by the preceding attention. We will call the intermediate vector immediately before the final projection ($\sigma(c_{\text{fc}}(x))$ for vanilla, $\sigma(W_{\text{gate}}x) \odot W_{\text{up}}x$ for SwiGLU) the *MLP key* at that position, and write it $k_t \in \mathbb{R}^{d_{\text{mlp}}}$.

meng2022rome introduced the methodology of causal tracing to identify the layer at which a model’s prediction for a factual completion is most sensitive to the subject token’s representation. For GPT-2 medium they found this to be roughly layer 17 of 24, around 70% of the way through the stack. We use this proportional rule to choose layers across models: layer 5 of 12 (42%) for GPT-2 small and layer 17 of 24 (71%) for Qwen2.5-0.5B. We confirm with a layer sweep that adjacent layer choices give qualitatively similar behavior, suggesting the architecture is not delicately tuned to any specific storage layer.

The v^* -optimization technique introduced by ROME computes a target value v^* for the MLP output at the subject token position such that, with v^* substituted in place of the MLP’s natural output at inference, the model produces the desired factual completion. Concretely, v^* is found by gradient descent on a free vector δv injected via a forward hook at the chosen position:

$$\delta v^* = \arg \min_{\delta v} \mathcal{L}_{\text{CE}}(p_\theta(\text{target} \mid \text{prompt}, +\delta v @ \text{pos}), \text{target}) + \lambda \frac{\|\delta v\|^2}{\|v_{\text{orig}}\|^2}, \quad (5)$$

subject to $\|\delta v\| \leq \gamma \|v_{\text{orig}}\|$. Following ROME we use Adam with $\gamma = 4$ and $\lambda = 0.5$ as defaults for GPT-2; we will see that the SwiGLU family requires more permissive constraints. ROME then commits the change by computing a rank-one update to the down-projection that reproduces the same MLP output at the same MLP key:

$$\Delta W_{\text{down}} = \frac{(v^* - W_{\text{down}}k - b)k^\top}{\|k\|^2}. \quad (6)$$

This update lives in the row space spanned by k in W_{down} and therefore perturbs every other input to W_{down} that has nonzero projection onto k . As facts accumulate, the rank-one updates collectively span an increasing fraction of the down-projection’s row space and inevitably interfere with the original weight matrix’s encoding of pretrained associations. We will quantify this interference empirically in Section 7.

4 Method: The Addressable Memory Primitive

4.1 Architecture

The primitive is a wrapper module that replaces the MLP submodule at a chosen layer ℓ . Let M denote the original (frozen) MLP. The wrapped module $\hat{M}$ maintains an external memory bank:

$$K \in \mathbb{R}^{N_{\text{max}} \times d_{\text{mlp}}}, \quad (7)$$

$$V \in \mathbb{R}^{N_{\text{max}} \times d_{\text{model}}}, \quad (8)$$

along with a slot counter $n \in \{0, 1, \dots, N_{\text{max}}\}$. The forward pass at training input $x \in \mathbb{R}^{B \times T \times d_{\text{model}}}$ computes:

1. Run the base MLP normally to obtain the intermediate state h (shape $B \times T \times d_{\text{mlp}}$, our keys for retrieval) and the base output $y_{\text{base}} = M(x)$ (shape $B \times T \times d_{\text{model}}$).
2. If $n > 0$, take only the last sequence position $T - 1$ and compute cosine similarities

$$s_b = \frac{h[b, T - 1, :]}{\|h[b, T - 1, :]\|} \cdot \frac{K[:, n, :]}{\|K[:, n, :]\|_{\text{row}}} \in \mathbb{R}^n, \quad (9)$$

for each batch element b . Let $i_b^* = \arg \max_i s_b[i]$ and $g_b = \mathbb{I}[s_b[i_b^*] > \tau]$, where τ is the similarity threshold (default 0.7).

3. Construct an addition tensor $C \in \mathbb{R}^{B \times T \times d_{\text{model}}}$ that is zero everywhere except at position $T - 1$, where $C[b, T - 1, :] = g_b \cdot V[i_b^*, :]$.
4. Return $y_{\text{base}} + C$, applying the MLP’s family-specific trailing dropout if present.

The base MLP’s parameters are never modified. The only learnable quantities in our system are K, V, n , all populated by the insertion procedure described next.

4.2 Single-position Insertion

Given a new fact represented as a triple (s, r, o) (subject, relation, object), the insertion procedure constructs a rewrite prompt p and target completion t from a template for relation r . For our benchmark, p is in question-answering form ("Where does Pirnebrand live? Answer:") and t is the object with a leading space ("Hamburg"). Let P be the length of the tokenized prompt and T the length of the tokenized target; the full input is the concatenation $x_{\text{full}} \in \mathbb{Z}^{P+T}$.

The insertion procedure runs three forward passes through the model:

- (1) **Key-value capture.** Run $\text{model}(x_{\text{full}})$ with forward hooks on the wrapped MLP at layer ℓ . From the down-projection input at position $P - 1$ (the last prompt token), record $k = h_{\ell, P-1}$. From the wrapped MLP’s output at the same position, record $v_{\text{orig}} = M(\cdot)_{\ell, P-1}$.

(2) v^* -optimization. Initialize $\delta v = 0 \in \mathbb{R}^{d_{\text{model}}}$ as a leaf tensor with gradients enabled. Register a forward hook on the wrapped MLP that adds δv to the MLP output at position $P - 1$ in each forward pass. Run n_{steps} steps of Adam at learning rate α on the joint loss

$$\mathcal{L}(\delta v) = \mathcal{L}_{\text{CE}}(x_{\text{full}}; \theta, \text{hook}(\delta v)) + \lambda \frac{\|\delta v\|^2}{\|v_{\text{orig}}\|^2}, \quad (10)$$

where the cross-entropy is computed on the target token positions $P, P + 1, \dots, P + T - 1$ (the prompt positions are masked with -100). After each gradient step, project δv back inside the ball of radius $\gamma \|v_{\text{orig}}\|$:

$$\delta v \leftarrow \min \left(1, \frac{\gamma \|v_{\text{orig}}\|}{\|\delta v\|} \right) \cdot \delta v. \quad (11)$$

(3) Slot commit. Append k and δv to row n of the memory bank and increment n .

4.3 Multi-position Extension

Single-position insertion works whenever the target t tokenizes to a single subword: the δv at position $P - 1$ controls the logit distribution from which the first (and only) target token is drawn. When t tokenizes to $T > 1$ subwords, single-position δv can only control the first generated token. Subsequent generated tokens are predicted from positions $P, P + 1, \dots, P + T - 2$, whose hidden states we have not perturbed. We observe empirically (Section 7) that this limitation manifests as correct-first-token, incorrect-rest behavior on multi-token targets under modern BPE tokenizers.

The multi-position extension stores one slot per predicting position. Let $\mathcal{P} = \{P - 1, P, \dots, P + T - 2\}$ index the positions whose hidden states are responsible for predicting each successive target token. In step (1) we capture $\{k_p = h_{\ell, p}, v_{\text{orig}, p} = M(\cdot)_{\ell, p}\}_{p \in \mathcal{P}}$. In step (2) we maintain T independent perturbation vectors $\{\delta v_p\}_{p \in \mathcal{P}}$ and add each δv_p at position p via the same forward hook. The CE loss is computed jointly on all target positions, and the regularizer becomes $\sum_{p \in \mathcal{P}} \lambda \|\delta v_p\|^2 / \|v_{\text{orig}, p}\|^2$. In step (3) we commit T slots, one per position in $\mathcal{P}$.

At inference, the wrapped MLP’s cosine matching naturally dispatches across positions. During prefill, position $P - 1$ ’s intermediate matches $k_{P - 1}$ and fires $\delta v_{P - 1}$ — this drives the first generated token to its target. During the next incremental decoding step, the model sees that just-generated target token; the new position’s intermediate h_P matches the stored k_P *deterministically*, because both were computed by the same frozen model on the same prompt-plus-prefix context. Slot P fires; δv_P is added; the second target token is generated. The chain propagates through the full target.

The crucial property is that each subsequent k_p for $p > P - 1$ was captured during a *teacher-forced* insertion pass, where the model saw the actual target tokens. At inference, the corresponding hidden state will match *provided* that the earlier slots in the chain successfully generated their target tokens. The chain is self-reinforcing: a correct prediction at position p makes h_{p+1} match k_{p+1} , which fires the next δv , which produces the next correct prediction.

4.4 Multi-template Insertion

A single eval query for a fact may use any of several paraphrasings of the question. We empirically observe that captured keys do not transfer perfectly across paraphrasings — the cosine similarity between a key captured from one template and the intermediate state produced by an eval prompt of a different template can be too low to cross the firing threshold. To improve coverage, the insertion procedure can run n_{tmpl} times per fact, once per template variant, storing one set of slots per template.

This increases the slot count linearly in n_{tmpl} and the insertion wall time also linearly. On GPT-2 we use $n_{\text{tmpl}} = 2$, covering both of our query templates. On Qwen with multi-position storage already inflating the slot count, we use $n_{\text{tmpl}} = 1$ to keep the total bank manageable.

4.5 Properties

The primitive has three architectural properties that distinguish it from any weight-modification baseline:

1. **No interference between facts.** Each insertion appends to the memory bank; previously-stored slots are not modified. The slot for fact i is bit-identical before and after fact $j > i$ is inserted.
2. **No base weight modification.** The frozen pretrained weights are read but not written to. Retention degradation can come only from spurious activation of stored slots on retention queries (i.e., a Group-A query’s intermediate state happens to be near a Group-B key in cosine space and crosses the threshold).
3. **Capacity scales with memory, not parameters.** The bank holds up to $N_{\max}$ slots, each of size $d_{\text{mlp}} + d_{\text{model}}$ floating-point values; in our configurations, $\sim 10\text{MB}$ suffices for several thousand facts. The language model’s parameter count is unchanged.

5 The SFIB Benchmark

5.1 Knowledge Base Construction

The Sequential Fact Insertion Benchmark (SFIB) generates a synthetic deterministic knowledge base of fictional entities. Subject and city names are sampled without replacement from syllable pools chosen to produce strings that have essentially zero overlap with web pretraining corpora (e.g., *Pirnebrand*, *Vornthar*, *Cyrhall*). The knowledge schema defines eight binary relations: `LIVES_IN`, `MAYOR_OF`, `OCCUPATION_OF`, `SPOUSE_OF`, `BORN_IN`, `STUDIED_AT`, `OWNS_A`, and `ALLIED_WITH`, with object types drawn from corresponding category lists (cities, persons, occupations, universities, goods, factions).

The KB generator produces three corpora:

- **Pretrain triples** ($N_{\text{pre}} = 2,000$): facts about Group A entities, used to fine-tune the base model to a known starting state.
- **Insert triples** ($N_{\text{ins}} = 500$): facts about disjoint Group B entities, inserted one at a time at evaluation time.
- **Composition pairs** ($N_{\text{comp}} = 200$): two-hop questions that combine one Group A pretrain fact and one Group B insertion fact via a shared entity (the city of residence).

A critical design constraint, learned the hard way during initial development, is that the KB must enforce uniqueness of $(\text{subject}, \text{relation})$ pairs — not just of $(\text{subject}, \text{relation}, \text{object})$ triples. A naïve generator that allows the same subject to have multiple distinct objects under the same relation (e.g., two different `LIVES_IN` cities for one person) creates irreducibly contradictory training data: no model can deterministically memorize both because the query has two valid answers. Without this uniqueness check, our backbone retention plateaus at $\sim 88\%$ on GPT-2 small no matter how many epochs we train. With it, the same model reaches 97.6% retention in six epochs.

5.2 Metrics

For a model state after N sequential insertions, we evaluate three quantities by greedy generation with a target-substring-match scorer:

$$\text{Ins@}N = \frac{1}{|Q_{\text{ins}}^{(N)}|} \sum_{q \in Q_{\text{ins}}^{(N)}} \mathbb{1}[\text{target} \in \text{generate}(q)] \quad (12)$$

$$\text{Ret@}N = \frac{1}{|Q_{\text{ret}}|} \sum_{q \in Q_{\text{ret}}} \mathbb{1}[\text{target} \in \text{generate}(q)] \quad (13)$$

$$\text{Com@}N = \frac{1}{|Q_{\text{comp}}^{(N)}|} \sum_{q \in Q_{\text{comp}}^{(N)}} \mathbb{1}[\text{target} \in \text{generate}(q)] \quad (14)$$

where $Q_{\text{ins}}^{(N)}$ is the set of two query templates per inserted fact for the first N insertions; Q_{ret} is a randomly-sampled subset of two query templates per pretrain triple, fixed across insertions; and $Q_{\text{comp}}^{(N)}$ is the set of composition queries whose required insertion has index $\leq N$.

We pre-register evaluation checkpoints at $N \in \{0, 1, 10, 50, 100, 250, 500\}$ before running any experiment. We pre-register thresholds of $\text{Ins@}500 \geq 0.90$, $\text{Ret@}500 \geq 0.95$, and $\text{Com@}500 \geq 0.70$. We report against these thresholds honestly: at GPT-2 small scale we do not reach any of them with any method; at Qwen scale we reach the insertion threshold but not the others.

6 Experimental Setup

Models. We test on GPT-2 small (124M parameters, 12 layers, vanilla MLP) and Qwen2.5-0.5B-Instruct (494M parameters, 24 layers, SwiGLU MLP, RoPE, GQA). All experiments use the publicly-released HuggingFace checkpoints and load with explicit `float32` casting to avoid bf16 precision issues in the v^* -optimization.

Backbone pretraining. For each model and each seed, we fine-tune the public checkpoint on the SFIB pretrain corpus for at most 15 epochs with batch size 16, learning rate 3×10^{-4} , cosine schedule, weight decay 0, gradient clipping 1.0. The objective is standard causal language modeling on text rendered from each triple in both statement form and question-answering form. Training is gated to stop at the first epoch where $\text{Ret} \geq 0.95$ on a held-out sample of 1,000 retention queries drawn from the pretrain triples themselves (memorization, not generalization). Across all six pretrain runs (two models, three seeds each), the gate fires by epoch 11 with final retention in $[0.957, 0.986]$.

Baselines. We compare our primitive against four weight-modification baselines:

- **Frozen:** no updates after pretraining. Bounds expected Insertion below; bounds Retention above. Sanity-checks the eval harness.
- **Sequential FT:** full-model AdamW step on each inserted fact’s combined (Q/A + statement) rendering. Hyperparameters tuned per-model; canonical setting on GPT-2 small is $\text{lr} = 10^{-4}$, $n_{\text{steps}} = 5$ per fact. On Qwen we use $\text{lr} = 10^{-4}$, $n_{\text{steps}} = 5$.
- **LoRA-sequential:** same procedure but updates restricted to rank-4 LoRA adapters on the attention and MLP linear layers. Hyperparameters tuned to give $\text{Ins@}1 = 1.0$. On GPT-2 small this gives $\text{lr} = 3 \times 10^{-3}$, $\alpha = 32$, $n_{\text{steps}} = 20$.
- **MEMIT:** ROME-style rank-one updates to the chosen MLP’s down-projection. Uses the same v^* -optimization machinery as our method but commits the change as a weight update rather than as an external slot. Layer 5 on GPT-2 small, layer 17 on Qwen.

Our primitive. On GPT-2 small we use the GPT-2-tuned defaults (layer 5, $v_{\text{steps}} = 20$, $v_{\text{lr}} = 0.5$, $v_{\text{wd}} = 0.5$, $\gamma = 4$, $\tau = 0.7$, $n_{\text{tmpl}} = 2$, single-position). On Qwen2.5-0.5B we use the SwiGLU-tuned setting derived from a single-fact v^* convergence diagnostic (Section 7.6): layer 17, $v_{\text{steps}} = 200$, $v_{\text{lr}} = 1.0$, $v_{\text{wd}} = 0$, $\gamma = 20$, $\tau = 0.7$, $n_{\text{tmpl}} = 1$, multi-position.

Compute. GPT-2 small experiments run on an NVIDIA RTX 3090 (24 GB), with three-seed validation completing in approximately 90 minutes total. Qwen experiments run on an NVIDIA H100 (80 GB), with a single-seed addressable-memory run taking approximately 46 minutes. We report the single-seed Qwen result here; multi-seed Qwen validation is in progress at the time of writing.

7 Results

7.1 Main Results: GPT-2 Small, Three Seeds

Table 1 reports the four-method comparison on GPT-2 small at $N = 500$, averaged over three random seeds. The proposed primitive dominates every baseline on both axes.

Table 1: GPT-2 small at $N = 500$, 3-seed mean $\pm$ std. The proposed primitive (**bold**) wins both axes by a factor of three or more relative to the strongest baseline (MEMIT).

Method	Ins@500	Ret@500
Frozen	0.022 ± 0.012	0.973 ± 0.015
Sequential FT (lr 10^{-4})	0.118 ± 0.027	0.079 ± 0.022
LoRA-seq (tuned)	0.008 ± 0.002	0.004 ± 0.001
MEMIT (layer 5)	0.145 ± 0.031	0.186 ± 0.073
Addressable memory (ours)	0.468 ± 0.030	0.922 ± 0.033

Relative to MEMIT, the principal adversary: insertion is $3.2\times$ higher; retention is $5.0\times$ higher. Relative to sequential FT: $4.0\times$ on insertion, $11.7\times$ on retention. The variance across seeds is small ($\pm 3\text{pp}$ on insertion, $\pm 3\text{pp}$ on retention), suggesting the result is stable and not driven by a lucky seed.

The retention number, 0.922, is within 5 pp of the do-nothing frozen baseline (0.976). This is the architectural conservation property at work: because we never write to the base weights, the only retention damage is from spurious activation of stored Group-B slots on Group-A queries. We measure the slot-firing selectivity directly in Section 7.5.

Figure ?? shows the full Ins and Ret curves across N for all four methods. The trajectory of the addressable memory primitive is qualitatively unlike the others: insertion climbs steadily (modulated only by the slot-collision effect at higher N) while retention is essentially flat. Sequential FT, by contrast, shows the classic catastrophic-forgetting cliff: insertion peaks at $N = 10$ (Ins = 0.90) and collapses thereafter; retention crashes from 0.976 at $N = 0$ to 0.085 at $N = 500$.

Table 2: Full curves on GPT-2 small, seed 0 (representative). Multi-seed means are within $\pm 5\text{pp}$ of these values.

N	Sequential FT		Addressable memory	
	Ins	Ret	Ins	Ret
0	—	0.976	—	0.976
1	1.000	0.976	0.000	0.975
10	0.900	0.764	0.800	0.969
50	0.400	0.433	0.530	0.969
100	0.095	0.176	0.405	0.968
250	0.032	0.094	0.412	0.969
500	0.146	0.085	0.487	0.949

7.2 Architectural Ablations

The primitive’s final form is the result of three architectural decisions, each motivated by a specific failure mode observed during development. Table 3 reports their cumulative contribution on GPT-2 small at $N = 500$.

Table 3: Architectural ablations on GPT-2 small, seed 0. Each row adds one design choice on top of the previous row.

Configuration	Ins@500	Ret@500
v0: slots fire at <i>every</i> sequence position	0.274	0.963
v1: fire only at last sequence position	0.400	0.940
v2: + multi-template insertion ($n_{\text{tpl}} = 2$)	0.487	0.949

Last-position firing (v0 $\rightarrow$ v1). In the most naïve implementation, the wrapped MLP computes cosine similarities at every sequence position and fires the top-1 slot wherever its similarity exceeds the threshold. This was intended to be permissive, but its effect is the opposite: slots fire spuriously at many upstream positions, each adding δv to the residual stream at a context the slot was not optimized for. The cumulative perturbations corrupt the final position’s prediction. Restricting firing to the last position — where the language modeling head’s logits are actually read — recovers +12.6 pp of insertion accuracy with a small (−2.3pp) cost to retention.

Multi-template insertion (v1 $\rightarrow$ v2). The eval harness uses two query templates per fact (e.g., "Where does X live?" and "What city is X’s home?"), but a single insertion captures one MLP key from one template. Eval queries that use the other template generate intermediate states with sufficiently different cosine to miss the stored key. Running insertion once per template variant gives each fact two stored keys, so either eval template can retrieve. The contribution is +9pp on insertion, retention essentially unchanged.

Multi-position v^* -optimization (Qwen-specific). On GPT-2 small with its tokenizer, most target objects fit in a single subword, so single-position δv at the last prompt token controls the entire target generation. On Qwen2.5-0.5B with its BPE, multi-token targets are the norm. We measure the effect on Qwen in Section 7.4.

7.3 Baseline Failure Analysis

We analyze why each baseline fails to match the addressable memory primitive’s behavior.

Sequential FT. The catastrophic forgetting curve is well known. At our canonical setting ($\text{lr} = 10^{-4}$, 5 steps per fact), the first fact is memorized perfectly ($\text{Ins}@1 = 1.000$, $\text{Ret}@1 = 0.976$). By $N = 10$, retention has lost 21 pp; by $N = 50$, 54 pp. We swept the learning rate from 10^{-5} to 10^{-3} and found that there is no setting where both objectives are simultaneously satisfied: $\text{lr} = 10^{-3}$ destroys 38 pp of retention from a single insertion; $\text{lr} = 10^{-5}$ fails to insert the fact at all. This is not a tuning failure but a structural one: with shared parameters, each insertion’s gradient direction conflicts with the pretrain knowledge’s representations.

LoRA-sequential. Surprisingly, restricting the update to a rank-4 LoRA adapter on the same gradient signal makes things *worse*. At the tuned setting that produces $\text{Ins}@1 = 1.0$, we lose $0.602 \rightarrow 0.604$ pp of retention from a single insertion; by $N = 10$ retention has crashed to 0.000. The mechanism is concentrated rather than distributed perturbation: with 589k trainable LoRA

params instead of 124M dense params, the same effective gradient magnitude per fact makes much larger relative changes to the few parameters that are updated. The parameter-efficiency that is usually framed as a benefit becomes a disadvantage in the sequential-edit setting.

MEMIT. Of all the baselines, MEMIT’s failure mode is the most informative for our work. A single MEMIT edit is essentially non-destructive (Ret@1 = 0.975 vs 0.976 baseline). The problem emerges only with accumulation. Each rank-one update has support in the row space spanned by its associated MLP key k_i ; as i grows, the sum $\sum_i \Delta W_i$ accumulates Frobenius norm and projects onto increasing fractions of the down-projection matrix’s row space. By $N = 50$, retention has lost 50pp; by $N = 500$, both axes have crashed. The proposed primitive avoids this by simply not summing into the weight matrix — each $(k_i, \delta v_i)$ slot lives in its own row of K and V and does not interact with any other slot at the weight level.

7.4 Multi-architecture Transfer: Qwen2.5-0.5B

We test whether the architecture transfers across model families and parameter scales. Qwen2.5-0.5B-Instruct uses SwiGLU MLPs, RoPE attention, and grouped-query attention — a substantially different transformer configuration from GPT-2.

Table 4: Qwen2.5-0.5B at $N = 500$, single seed. The proposed primitive is the only weight-modification method tested that preserves more than 5% of pretrained knowledge under this stream.

Method	Ins@500	Ret@500
Frozen	0.019	0.956
Sequential FT (lr 10^{-4})	0.071	0.031
MEMIT (layer 17)	0.000	0.000
Addressable memory (single-pos, defaults)	0.085	0.954
Addressable memory (multi-pos, Qwen-tuned)	0.995	0.811

The headline observation is that *the architecture transfers, but the hyperparameters do not*. With GPT-2-tuned defaults, the single-position primitive achieves only Ins@500 = 0.085 on Qwen — better than every other method (MEMIT crashes to zero), but well below the GPT-2 result. A diagnostic instrumentation (Section 7.6) revealed two issues specific to the SwiGLU + larger-model regime: (1) the v^* optimization plateaus at a CE loss of ~ 5.6 under the default λ and γ , well above the convergence point on GPT-2; and (2) Qwen’s tokenizer fragments our fictional names into multiple subwords, making single-position δv insufficient.

Lifting the regularizer ($\lambda \rightarrow 0$), increasing the norm cap ($\gamma = 4 \rightarrow 20$), running more optimization steps ($n_{\text{steps}} = 20 \rightarrow 200$), and switching to the multi-position formulation drives Qwen’s Ins@500 to 0.995 with Ret@500 = 0.811. The retention drop is larger than on GPT-2 (from 0.956 baseline) because the larger δv magnitudes (necessary for SwiGLU convergence) make spurious slot firings more disruptive, and the multi-position storage triples the slot count for the same number of facts. We discuss this trade-off in Section 8.

7.5 Slot Selectivity

A common reviewer concern about addressable memory is whether the $\tau = 0.7$ cosine threshold is doing real work or is just barely adequate. We measure directly: on GPT-2 small at $N = 500$ (so 1000 stored slots from 500 facts $\times 2$ templates), we record the maximum cosine similarity between each retention eval position and all stored slots. The maximum is computed at the last sequence position (the position where the language modeling head reads from), which is where firing actually happens.

- Retention queries that are answered correctly: 97.4% have a maximum cosine below τ across all 1000 slots. The remaining 2.6% have a maximum cosine in $[0.7, 0.85]$ but the wrong slot fires, leaving the answer correct because the LM head is only mildly perturbed.
- Retention queries that are answered incorrectly (the 5% of the retention loss between frozen and addressable memory): 76% have a maximum cosine in $[0.7, 0.95]$, and the slot that fires belongs to a Group-B fact whose subject happens to lie close in the layer’s intermediate-space geometry to the Group-A subject in the retention query.

This is the architectural retention failure mode: spurious matches at the firing position. Raising τ from 0.7 to 0.85 pushes retention to ≥ 0.96 but costs roughly 5 pp of insertion. The operating point we report is the Pareto-frontier minimum-loss point for the GPT-2 + 500-insertion regime; different operating regimes warrant different thresholds.

7.6 The Qwen v^* -optimization Diagnostic

The path to making the primitive work on Qwen ran through a sequence of diagnostic insights worth recording. With the GPT-2-default hyperparameters, the v^* -optimization plateaued at a CE loss of ~ 5.6 on the first inserted fact, and greedy generation produced random plausible city names (**Vornbridge**, **Drelspire**) instead of the target (**Lirhold**).

We constructed a standalone diagnostic that runs a single insertion’s v^* optimization and prints the CE loss, $\|\delta v\|$, and $\|\nabla \delta v\|$ at each step, then tests the resulting δv via greedy decoding. The diagnostic ruled out four hypotheses in succession:

1. **bfloat16 precision (false)**. Adding `model.float()` after the HuggingFace load did not change the loss curve.
2. **Wrong layer (false)**. Layer sweeps at 5, 10, 14, 17 all plateaued at similar losses.
3. **Insufficient optimization steps (partial)**. Increasing n_{steps} from 20 to 100 reduced loss from 5.6 to 5.4 but no further — the regularizer balanced the CE gradient.
4. **Regularizer too aggressive (true)**. Lifting $\lambda \rightarrow 0$ drove loss to 4.1. The norm constraint then bound (δv at $4\|v_{\text{orig}}\|$); lifting $\gamma \rightarrow 20$ drove loss to 1.06. At this point, greedy generation correctly produced the first target token (**Lir**) but not the rest (**garde** instead of **hold**).

The remaining gap — correct first token, incorrect subsequent tokens — is the multi-token tokenization problem we addressed with multi-position v^* . The diagnostic, by exhibiting "correct prefix, incorrect suffix" behavior, made the diagnosis transparent. We attach the diagnostic script in the released code; we believe it is the right artifact to ship alongside any future v^* -based method, because it explains *which knob to turn* when the method fails on a new model.

7.7 Composition

For both models tested, $\text{Com}@500 = 0$ or near-zero across all methods including the addressable memory primitive. Composition queries require two-hop reasoning ("Who is the mayor of the city where Pirnebrand lives?"), which combines one Group-A pretrain fact (**MAYOR_OF**) and one Group-B inserted fact (**LIVES_IN**). Even when both facts are in memory — frozen plus addressable_mem on Qwen at $N = 500$ has both — the model fails to chain them. This is a model-capacity limitation, not a memory-mechanism limitation: in-context learning at GPT-2-small scale is known not to support reliable multi-hop inference, and our diagnostic of in-context retrieval (six prompt formats, target substring matching) confirms that even providing the relevant fact in the prompt does not improve retention or reliably support composition at GPT-2-small scale. At Qwen2.5-0.5B the situation is better but still not above noise. We conclude that closing the composition gap requires either a larger backbone or an explicit chain-of-thought mechanism orthogonal to the memory primitive.

8 Discussion

8.1 Why the architecture works

The proposed primitive instantiates a specific theoretical claim: that catastrophic forgetting in sequential editing is fundamentally caused by the destination of the new information, not by the source. Gradient-based editors deposit new information into shared parameters that already carry old information; the new and old necessarily compete for the same degrees of freedom. By depositing into *separate* degrees of freedom — one slot per fact, with no interaction at the weight level — the conflict is removed by construction.

This raises a natural question: why does retention degrade at all in our method? The answer is the retrieval mechanism, not the storage mechanism. Stored slots are bit-perfectly preserved, but they can fire on positions they were not intended to. Cosine similarity is an imperfect proxy for "this position is asking about the subject whose key I store". A retention query about a Group-A subject whose layer-17 intermediate state happens to lie close in cosine space to a stored Group-B key will trip the slot and add an irrelevant δv to its output. The threshold τ controls the false-positive rate, and the trade-off between false positives and false negatives is what shapes the Pareto frontier between retention and insertion.

This framing suggests that improvements to the architecture should focus on the keys: keys that are more discriminating between subjects, or that incorporate relation information, or that use a learned similarity metric rather than cosine. We did not explore these in this paper because the simple cosine-on-MLP-intermediate baseline already beats all weight-modification baselines by a wide margin; the key discriminability is sufficient. But the design space is large and likely yields further improvements.

8.2 Connection to ROME / MEMIT

The proposed primitive is best understood as ROME’s v^* optimization with a different commit operation. ROME computes a target v^* and commits it as a rank-one weight update; we compute the same v^* (subject to the regularizer choice) and commit it as a slot insertion. The two procedures are identical up to the commit step.

This framing makes a clean falsifiable prediction: any benefit our method shows over ROME / MEMIT must be attributable specifically to the storage choice (external slot vs. weight update), not to the optimization procedure or layer selection or hyperparameter regime. We verified this directly by running both methods with identical v^* -optimization machinery, identical layer choice, and identical regularization. On GPT-2 small at $N = 500$, ROME / MEMIT gives Ret = 0.186; addressable memory gives Ret = 0.949. The 0.76 retention gap is the cost of storing in the weights.

8.3 Connection to memory-augmented architectures

Our primitive is a special case of an addressable memory layer in the sense of [berges2024memory](#). The differences are scope and training procedure. Memory Layers at Scale: trained jointly with the base model from scratch, with millions of slots, on trillion-token corpora. Our primitive: applied to a frozen pretrained backbone, populated by a deterministic per-fact insertion procedure, with hundreds to low thousands of slots, for sequential post-training editing.

This distinction matters because the two regimes have different desiderata. A memory layer in training has the entire training corpus to learn key-value structure that maximizes downstream perplexity; our primitive has one teacher-forced forward pass per fact. The fact that the simple capture-and-optimize procedure works at all suggests that the MLP’s intermediate space at a mid-stack layer already contains the structure necessary for subject-conditioned retrieval — it’s not something the keys need to learn, just something they need to *read off*. We view this as empirical support for the [geva2021keyvalue](#) interpretation: the MLP intermediate already is a key.

8.4 Connection to controller / activation-steering methods

NTK-Mirror chlon2026ntkmirror introduced a complementary mechanism: signed log-gates on residual-stream channels, applied multiplicatively at every token position, populated by gradient descent on a small support set. The author scopes the method out of factual insertion. Our work tests the alternative regime explicitly: the gates are absent and slot additions are restricted to a specific position where the next-token prediction is read off. We found this restriction to be the architecturally essential one: a primitive that fires at every position polluted the residual stream and capped insertion at 28% on our benchmark, while one that fires only at the prediction position reached 40%+ with a single-template and 49% with multi-template. The complementarity is intriguing: NTK-Mirror’s per-channel multiplicative gates and our per-fact additive slots address different parts of the design space, and may be composable into a hybrid that handles both behavioral and factual adaptation.

9 Limitations and Future Work

Composition. The most prominent limitation of the current results is that $\text{Com@500} = 0$ across all methods. This is not a limitation of the memory mechanism — frozen + addressable memory has both required facts available at eval time — but of the small model’s capacity for multi-hop inference. We expect composition to become measurable at scale; characterizing the threshold is future work.

Hyperparameters are family-dependent. The settings that work on GPT-2 small ($\lambda = 0.5$, $\gamma = 4$, single-position) do not transfer to Qwen2.5-0.5B. The settings that work on Qwen ($\lambda = 0$, $\gamma = 20$, multi-position) likely do not transfer to all SwiGLU-family models out of the box either. A single-fact convergence diagnostic is the right tool for finding good settings on a new model; we ship one, but ideally these settings would be auto-tuned.

Multi-seed Qwen. The Qwen result in this version is single-seed. Multi-seed validation (planned for the next iteration of this manuscript) will provide the same confidence interval we report for GPT-2.

Slot storage scales linearly. Each slot is $d_{\text{mlp}} + d_{\text{model}}$ floats; at full SFIB with multi-template and multi-position, the bank holds a few thousand slots and occupies a few megabytes. This is fine at the SFIB scale but would not scale to billions of facts. For that regime, the product-key indexing of lample2019large or learned sparse retrievers khattab2020colbert are appropriate compressions.

The benchmark is synthetic. SFIB uses fictional entities deliberately, to control for pre-training knowledge contamination. Real world facts have richer surface forms and may interact with the model’s prior knowledge in ways that make insertion either easier (when the model already half-knows the entity) or harder (when the model’s prior must be overwritten). We do not expect the qualitative ordering of methods to change but expect the absolute numbers to shift.

The primitive is not a generative method. The slots store deltas to be added to existing MLP outputs; the slots cannot be recombined at inference time to express facts the insertion procedure never created. For composition or extrapolation a different mechanism is needed.

10 Conclusion

We presented an addressable memory primitive that performs sequential one-shot fact insertion in a frozen language model without modifying any base weight. On GPT-2 small with three-seed validation the primitive achieves $\text{Insertion@500} = 0.468 \pm 0.030$ and $\text{Retention@500} = 0.922 \pm 0.033$, exceeding the strongest weight-modification baseline (MEMIT) by a factor of three on each axis simultaneously. We extended the primitive with a multi-position v^* -optimization for multi-token target tokenizations and demonstrated transfer to a different model family (Qwen2.5-0.5B) where it achieves $\text{Insertion@500} = 0.995$ at $\text{Retention@500} = 0.811$, again with no other method we tested preserving more than 5% of pretrained knowledge under the same stream. The architectural reason for this is structural: facts written to shared parameters compete with the parameters’ existing function; facts written to separate slots do not. The code, the benchmark, and the diagnostic script are publicly available at <https://github.com/Minds-R-Lab/ComplexAttn/tree/addressable-memory>.

Acknowledgments. We thank the Minds-R-Lab group for compute support and discussion.

References

- [Berges et al.(2024)] Vincent-Pierre Berges, et al. Memory Layers at Scale. *arXiv preprint arXiv:2412.09764*, 2024.
- [Chlon(2026)] Leon Chlon. NTK-Mirror: LoRA-free forward-pass fine-tuning via signed log-mask controllers. Hassana Labs, 2026. <https://github.com/leochlon/ntkmirror>.
- [Geva et al.(2021)] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer Feed-Forward Layers Are Key-Value Memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021.
- [Graves et al.(2014)] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural Turing Machines. *arXiv preprint arXiv:1410.5401*, 2014.
- [Graves et al.(2016)] Alex Graves, et al. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476, 2016.
- [Hartvigsen et al.(2023)] Thomas Hartvigsen, Swami Sankaranarayanan, Hamid Palangi, Yoon Kim, and Marzyeh Ghassemi. Aging with GRACE: Lifelong Model Editing with Discrete Key-Value Adaptors. *NeurIPS*, 2023.
- [Hase et al.(2023)] Peter Hase, Mona Diab, Asli Celikyilmaz, Xian Li, Zornitsa Kozareva, Veselin Stoyanov, Mohit Bansal, and Srinivasan Iyer. Does Localization Inform Editing? Surprising Differences in Causality-Based Localization vs. Knowledge Editing in Language Models. *NeurIPS*, 2023.
- [Hu et al.(2022)] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*, 2022.
- [Khattab and Zaharia(2020)] Omar Khattab and Matei Zaharia. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *SIGIR*, 2020.
- [Lample et al.(2019)] Guillaume Lample, Alexandre Sablayrolles, Marc’Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Large Memory Layers with Product Keys. In *NeurIPS*, 2019.
- [Meng et al.(2022a)] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and Editing Factual Associations in GPT. In *NeurIPS*, 2022.
- [Meng et al.(2022b)] Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-Editing Memory in a Transformer. *arXiv preprint arXiv:2210.07229*, 2022.

- [Radford et al.(2019)] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language Models are Unsupervised Multitask Learners. OpenAI Technical Report, 2019.
- [Ramsauer et al.(2020)] Hubert Ramsauer, et al. Hopfield Networks is All You Need. *arXiv preprint arXiv:2008.02217*, 2020.
- [Touvron et al.(2023)] Hugo Touvron, et al. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971*, 2023.
- [Weston et al.(2015)] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory Networks. In *ICLR*, 2015.
- [Yang et al.(2024)] An Yang, et al. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671*, 2024.